

07 Role-based access control, Database internals

▼ Overview of Access Control

Access Control Mechanisms

Discretionary Access Control (DAC):

- Each object has an owner (the role that created it), who **possesses the authority** to grant or revoke access to that object.
- OneDrive or Google Docs. If **you** create it, **you choose** who can edit or view

Role-Based Access Control (RBAC):

- Privileges are **assigned to roles rather than directly to users**. Users gain permissions **through the roles assigned** to them, promoting scalable and manageable access control.

Key concepts RBAC:

Securable Object:

- Entities such as **databases, schemas, tables, and views** to which access can be granted.

Role:

- A **collection of privileges** that can be assigned to users or other **roles**, **facilitating hierarchical permission structures**.

Privilege:

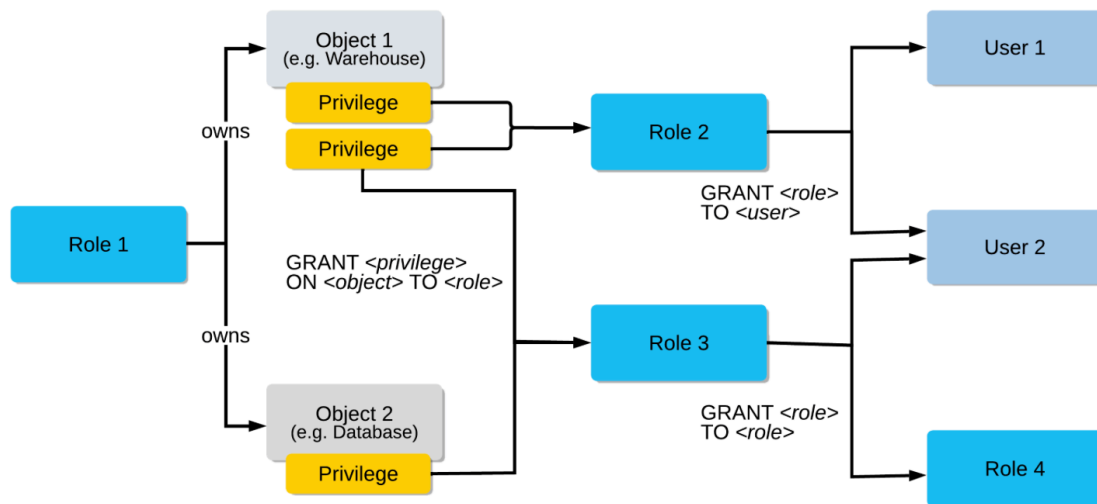
- Specific **rights assigned to roles**, defining permissible operations on securable objects (e.g., SELECT, INSERT).

User:

- An **individual** or **system entity** authenticated within Snowflake, **granted roles** to determine **access levels**.

Role Hierarchy and Privilege Inheritance:

- Roles can be **organized hierarchically**, allowing **child roles to inherit privileges from parent roles**.
- This structure enables **efficient management of permissions** across complex organizational structures.

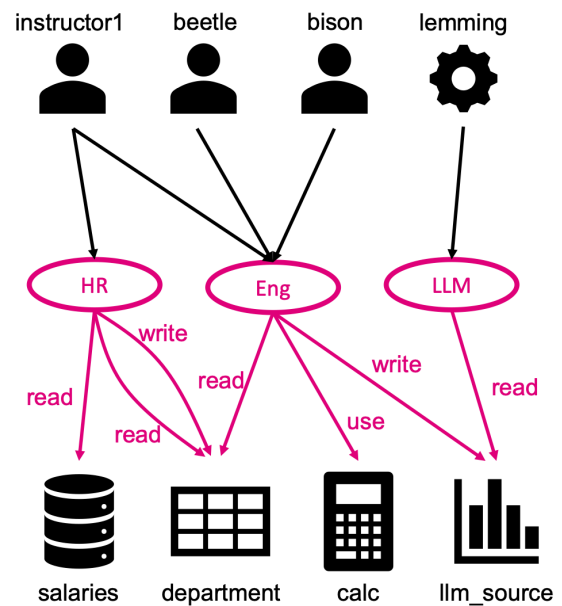


Managing Privileges:

- **Granting Privileges:** Roles with the **appropriate ownership or administrative rights** can **grant specific privileges** on objects to other roles using the **GRANT** command.
- **Revoking Privileges:** Similarly, the **REVOKE** command **allows roles to remove previously granted privileges**.

```
create role hr;
create role eng;
create role llm;
```

```
-- create "securable objects"
create table salaries(
employee_id int, salary double);
create table department(
employee_id int, department string)
create function calc(a int, b int)
returns int as $$a + b$$;
create table llm_source(data string)
```



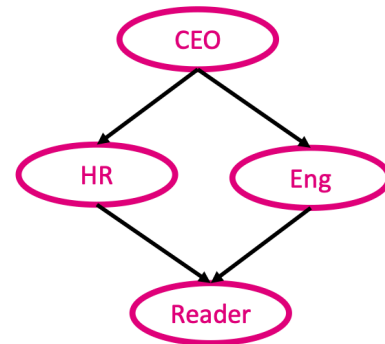
```
-- grant privileges to roles
grant select on table salaries to role hr;
grant select on table department to role hr;
grant insert on table department to role hr;
grant update on table department to role hr;
grant select on table department to role eng;
grant usage on function calc(int, int)
to role eng;
grant insert on table llm_source to role eng;
grant update on table llm_source to role eng;
grant select on table llm_source to role llm;
```

```
use role accountadmin;
grant role hr to user instructor1;
grant role eng to user instructor1;
grant role eng to user beetle;
```

```
grant role eng to user bison;  
grant role llm to user lemming;
```

Role Hierarchy

```
-- role hierachy  
create role ceo;  
create role reader;  
  
grant role hr to role ceo;  
grant role eng to role ceo;  
  
grant role reader to role hr;  
grant role reader to role eng;
```



▼ Storage Hardware and Physical Database Design

The Storage Hierarchy

Primary Storage (volatile):

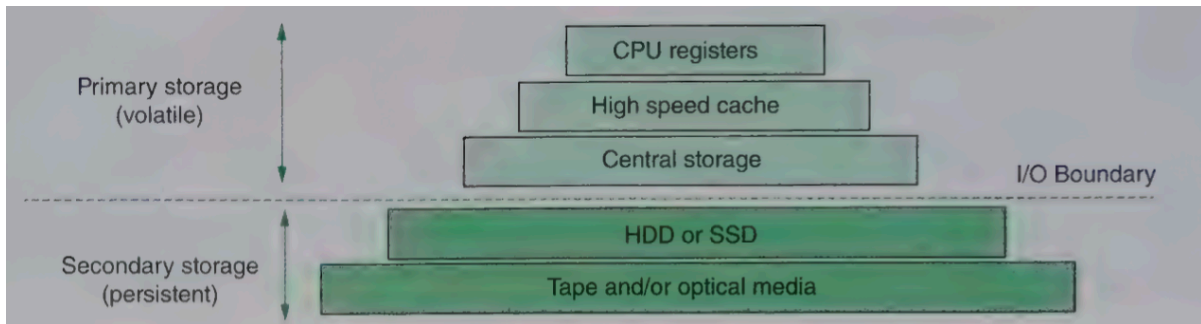
- Includes **CPU registers, cache memory, and RAM.**

Secondary Storage (persistent):

- Includes **Hard Disk Drives (HDDs)** and **Solid-State Drives (SSDs)**
- **Retain data even when powered off.**

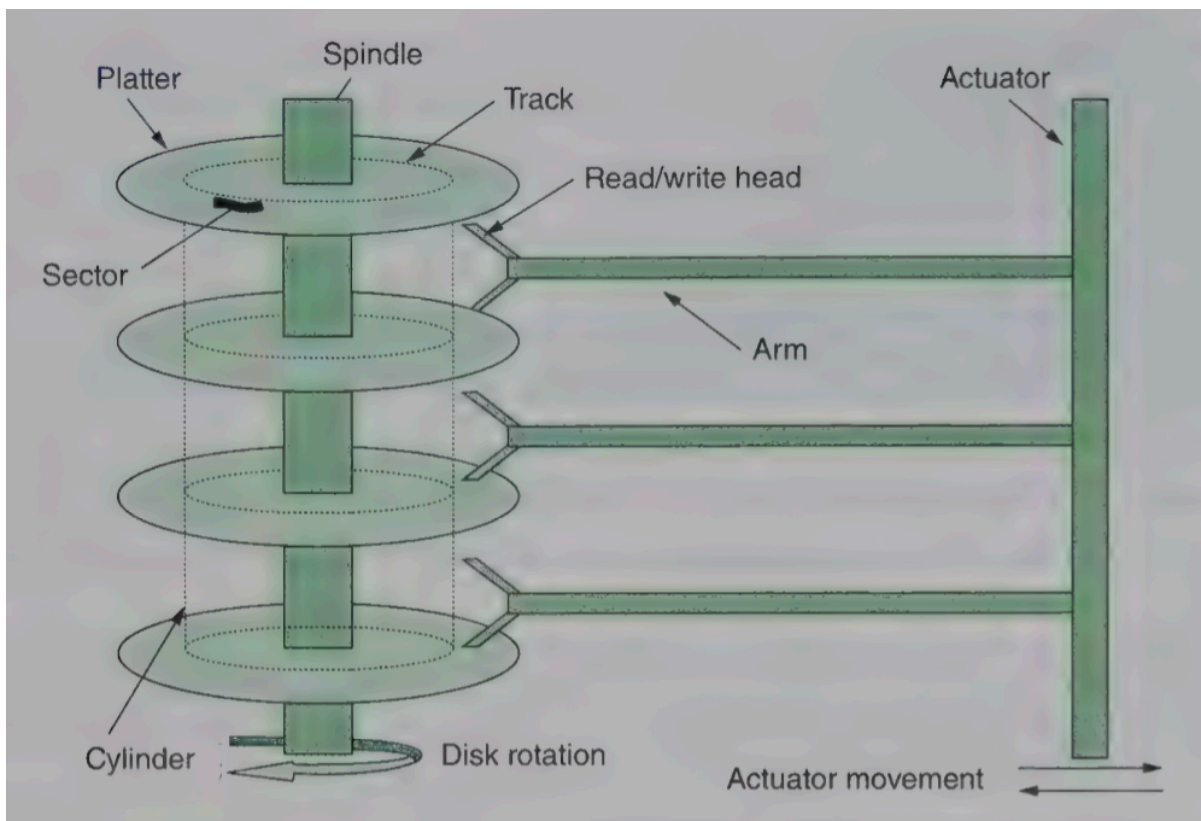
I/O Boundary:

- The **division between primary and secondary storage**
- Highlighting the slower access speed of secondary storage.



Internals of Hard Disk Drives (HDDs)

- HDDs use **magnetic platters** organized in **concentric circular tracks**, divided into **sectors**.
- **Seek Time:** The **time taken for the actuator to move to the correct track**.
- **Rotational Delay:** The **time for the desired sector to come under the read/write head**.
- **Transfer Time:** The **actual data reading/writing process**.

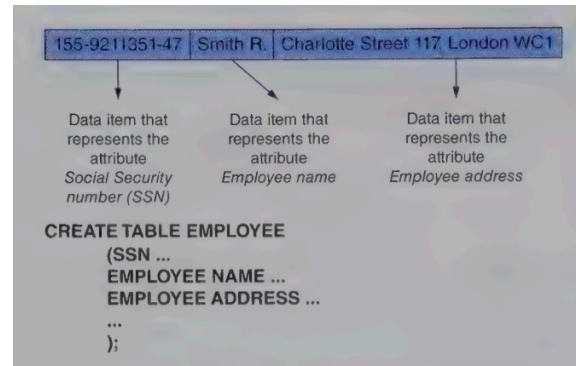


▼ Record Organization

Record Organization Strategies

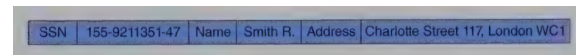
Relative Location:

- Stores **attributes adjacently in memory**.
- Efficient in terms of **storage space** but **inflexible for variable-length records**.



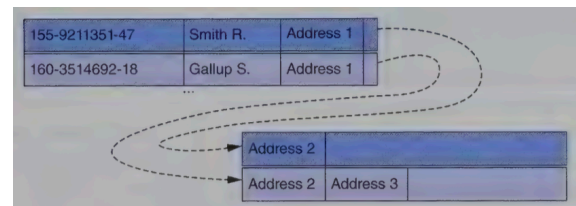
Embedded Identification:

- Stores **attribute names with values**.
- More **flexible but requires additional storage**.



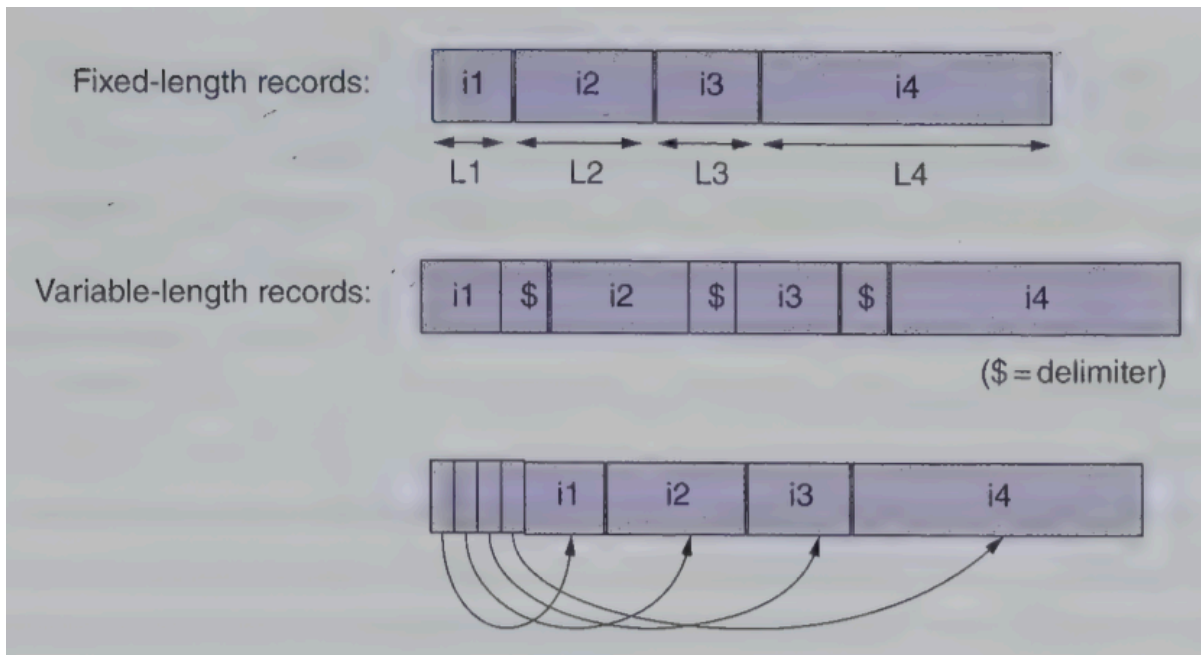
Pointers & Lists:

- Uses **pointers to link** variable-length records.
- Useful for **linked lists** and **overflow management**.



Fixed-length vs. Variable-length records

- Fixed-length records are **easy to index but waste space** if fields are empty.
- Variable-length records require **metadata for organization** (e.g., delimiters or pointers).



▼ File Organization

Introductory Concepts

Search Key:

- A **value** used for **retrieval**
- (e.g., primary key, index key).

Primary File Organization:

- Directly **affects the physical layout** of records.

Secondary File Organization:

- Uses **indexes** to **optimize search efficiency**.

Heap File Organization

- **Unordered file structure** where **new records are appended at the end**.
- Efficient for **insert-heavy operations** but slow for searches.

Sequential File Organization

- Stores **records in sorted order**.
- Allows **efficient range queries** but is expensive for inserts.

Example:

- If a **customer database** is sorted by CustomerID, searching for ID **0285719** is efficient but **inserting a new customer requires reordering**.

Random File Organization (Hashing)

- Uses **hash functions** to **compute storage** addresses.
- Fast for **retrieving records using exact match keys**.
- **Challenge**: Collisions occur when multiple keys map to the same location.

Example:

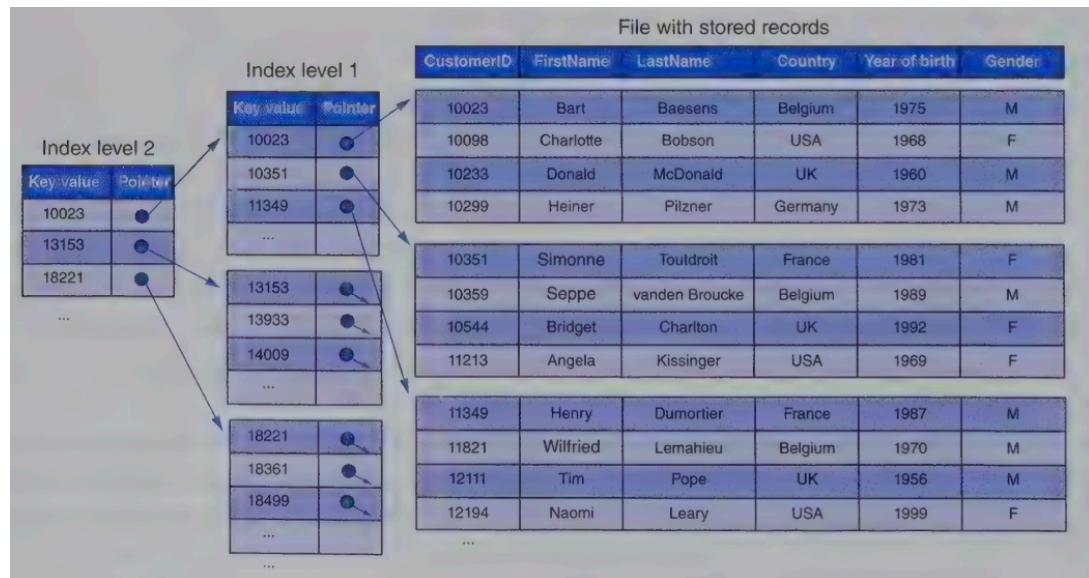
- **Hashing a customer's ID**:
 - `hash(CustomerID) → Storage Address`
 - If multiple **IDs hash to the same address, collision handling is needed** (e.g., chaining or linear probing).

Indexed Sequential File Organization

- Combines **sequential organization** with **indexing**.
- Uses **primary indexes, clustered indexes, and multi-level indexes** to improve search speed.

Example:

- A **multi-level index** (similar to a book's table of contents) helps locate data efficiently:
 - **Level 1**: Points to **Level 2** indexes.
 - **Level 2**: Points to **actual records**.



List Data Organization (Linear and Nonlinear Lists)

Linear Lists:

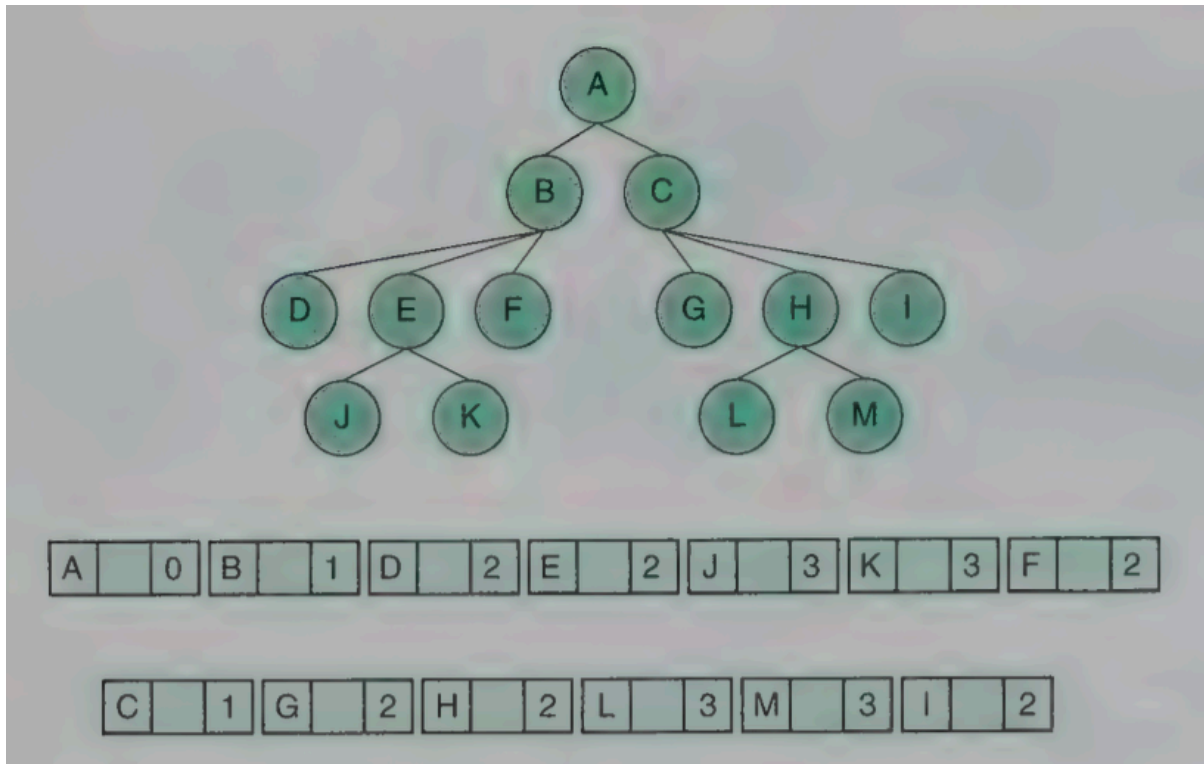
- Sequentially stored records (e.g., singly linked lists).

Tree Data Structures:

- Hierarchical storage, used in **B-Trees** and *BTrees*.

Example:

- A **linked list** structure is used for overflow records:
 - If an index page is full, a new page is linked to it rather than restructuring the entire file.

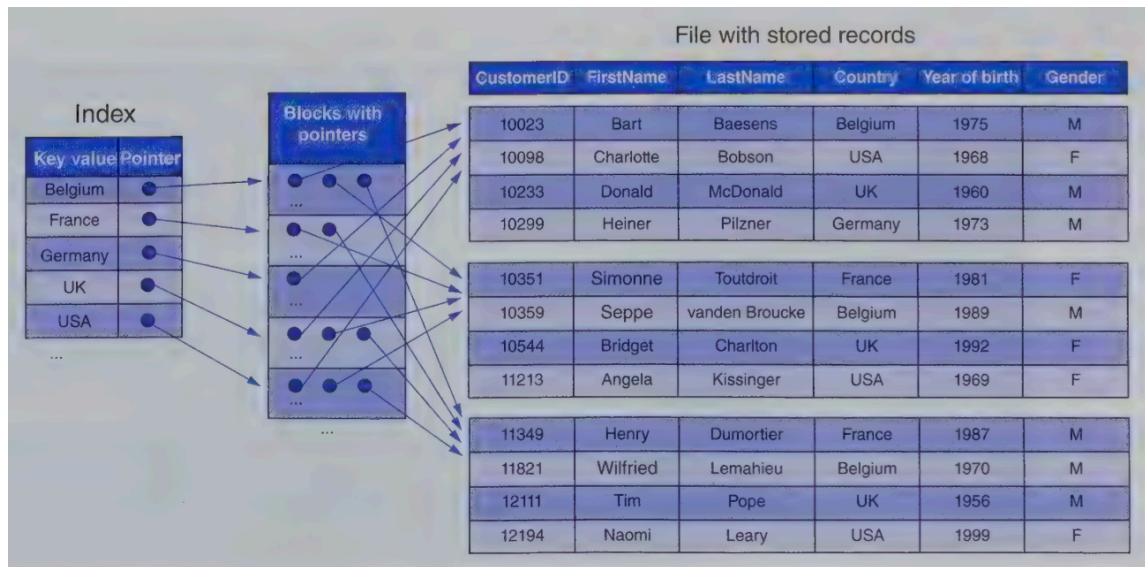


Secondary Indexes and Inverted Files

- **Secondary Indexes** allow searches on non-primary keys.
- **Inverted Files** are used in search engines for keyword lookups.

Example:

- A **secondary index** on "Customer Name" helps retrieve records **faster** even though the primary file is organized by "Customer ID."



Exercises